

DLP Protocol

Programmer Manual

The Drawing-Like Programming wire protocol, in full.

V1.0.0



Engineered For Tech Digestion

www.shanon-technologies.com

Contents

1	About this manual	3
1.1	Conventions	3
2	Orientation: what the protocol is doing	4
3	The protocol	5
3.1	Scope	5
3.2	Transport Model	5
3.3	Protocol Modes	6
3.3.1	Configuration Mode	6
3.3.2	Sampling Mode	6
3.4	Wire Encoding	7
3.5	Configuration Mode Frames	7
3.5.1	1. Header Frame	7
3.5.2	2. Body Stream	8
3.5.3	3. Response Poll Frame	9
3.5.4	4. Response Frame	9
3.6	Function Classes	10
3.6.1	Create range, 0x0000 to 0x0FFF	10
3.6.2	Link range, 0x1000 to 0x1FFF	18
3.6.3	Manage Range	18
3.7	Sampling Mode Commands	20
3.7.1	Sampling Mode Opcodes	20
3.7.2	Write Remote Inputs	21
3.7.3	Read Scalar Probes	21
3.7.4	Read Vector Probes	21
3.7.5	Switch Back To Configuration Mode	21
3.8	Expected Board State Machine	22
3.8.1	Reset and Selection	22
3.8.2	Configuration State Machine	22
3.8.3	Sampling State Machine	23
3.9	Host-Library Behavior	23
3.9.1	Sequence Numbers	23
3.9.2	Error Handling	23
3.9.3	Monitoring Interface	24

3.9.4 Multi-Board Sessions	24
3.10 Practical Transaction Summary	24
3.10.1 Configure Design	24
3.10.2 Stream Data	25
3.11 V2 Protocol Extensions	25
3.11.1 V2 Handshake	25
3.11.2 V2 Batch Upload	26
3.11.3 V2 Auto-Push Frame	27
3.11.4 V2 Flow Control	28
3.12 Protocol Limits	28
3.12.1 Manage range, 0x2000 and above	29
3.13 Parameter type identifiers	31
3.14 Design status values	31
3.15 The 3-second configuration-idle watchdog	32
3.16 Command ordering	33
3.17 Mode transitions	33
3.18 What the protocol does not protect you from	33
4 Implementation checklist	34
5 Glossary	35

Chapter 1

About this manual

This manual describes the **DLP (Drawing-Like Programming) protocol** at the level of detail a programmer needs to implement against it directly: frame layouts, field widths, byte order, the command set, and the sequencing rules that are not visible from any single frame.

Who it is for. Someone writing host software that talks to a PAPAYA VL board without using the supplied C++ or Python libraries – a different language, a constrained target, a test harness, or a protocol analyser. If you are writing an ordinary host program, use the libraries and read *PAPAYA Libraries – C++ and Python Reference* instead; everything here is already done for you there.

What it assumes. Comfort with byte-level serial protocols, two's-complement integers, IEEE-754 floating point and CRC. It does not assume any knowledge of the PAPAYA Studio application.

What it does not cover. The internal firmware architecture of the board, and the electrical characteristics of any interface. For the latter see *PAPAYA VL – Datasheet*.

1.1 Conventions

Convention	Meaning
<code>0x1234</code>	hexadecimal literal
<i>little-endian</i>	least-significant byte first, on the wire
host	the device driving the board
board	the PAPAYA VL
<i>configuration mode</i>	the structured, acknowledged command mode
<i>sampling mode</i>	the short streaming mode entered after StartSampling

All multi-byte integer fields are little-endian unless a field explicitly says otherwise. All float fields are IEEE-754 binary32, little-endian.

Chapter 2

Orientation: what the protocol is doing

A PAPAYA VL board does not run a program you compile. It runs a **block diagram** that the host *describes* to it, block by block and link by link, and then starts.

That shape explains almost every design decision in the protocol:

1. **Configuration is a sequence, not a document.** There is no single “here is my diagram” frame. The host issues one command per block and one per link, each acknowledged, in an order the board can execute.
2. **Order is load-bearing.** A link may only reference blocks that already exist, and the discretisation of a continuous block reads the sampling frequency *as it stands when that block is created*. Sending the same commands in a different order produces a different design, not an error.
3. **After StartSampling the protocol changes shape.** Structured, polled, CRC-checked frames give way to short streaming commands, because the board is now executing in hard real time and cannot afford the round trips.

The typical lifetime of a session is therefore:

```
DesignInit  -> Create x N  -> Link x M  -> StartSampling
                                         |
                                         write inputs / read probes
                                         |
                                         SwitchBackToCfg
```

Note

The board is configured, not programmed. There is no firmware to build and no toolchain to install for the board itself. That is why the protocol carries block descriptions rather than code.

Chapter 3

The protocol

3.1 Scope

It is the protocol used between a host and a PAPAYA board:

- PC
- Arduino
- Raspberry Pi
- STM32
- any other platform that provides the required PAPAYA transport adapters

This document does not describe any unrelated external-firmware protocol. It documents the actual host-to-PAPAYA protocol used by the current PAPAYA library.

3.2 Transport Model

- Canonical physical transport in the core implementation: SPI
- Default SPI mode: mode 0
- Bit order: MSB first
- Default SPI clock configured by the library: 8 MHz
- Chip select: software-driven, active low on direct SPI targets
- Target selection on direct SPI targets: one shared CS plus two board-select GPIO lines

Adapter note:

- on direct SPI targets, the host drives these signals itself
- on USB bridge targets, the DLP byte stream is still carried to the PAPAYA board, but USB endpoint selection is outside DLP and the reference desktop host does not use PAPAYA board-select semantics

The DLP frame format itself is independent from the host-side bridge protocol.

The board ID is not part of the byte stream. For direct SPI targets, the host selects the target board out of band by driving:

- PP_CS_PIN
- PP_ID_PIN_1
- PP_ID_PIN_2

Those three, and PP_RESET_PIN, are Arduino pin numbers, and they default to the positions the PAPAYA board's shield connector presents: D10 chip select, D8 and D9 board select, D2 board reset. A host whose own connector matches that outline needs nothing further.

Each of the four can be overridden. Define it before including the library header, or from the build (for example `-D PP_CS_PIN=5`), and the default is left alone. This matters on hosts where the Arduino pin number carrying a given shield pad is not the same number, which is the usual case on a board that provides the shield outline through an adaptor: taking the defaults there would drive four pins that are not connected to the PAPAYA board, and on some parts those numbers belong to the host's own memory bus. The clock, data in and data out lines are not in this list, because they belong to the `SPIClass` passed to `begin()` and every host routes them itself.

The library currently supports board IDs 0..3. On direct SPI targets, the board-select lines are driven with the inverted board ID before each SPI frame. On USB targets, the constructor still receives a board ID for API uniformity, but the USB transport does not use it as a hardware selector.

3.3 Protocol Modes

The protocol has two logical modes.

3.3.1 Configuration Mode

Used to:

- create blocks
- link blocks
- send manage commands such as `DesignInit`, `StopDesign`, `StartSampling`, `StopSampling`, `EnableRemoteInput`, and `SetMonInterface`

Configuration mode uses structured commands with:

- a fixed-size header
- a chunked command body
- a polled response frame

3.3.2 Sampling Mode

Used after `StartSampling`.

Sampling mode uses short streaming commands with no structured header:

- write remote inputs

- read scalar probes
- read vector probes
- switch back to configuration mode

3.4 Wire Encoding

- Integer fields are little-endian.
- float values are transmitted as IEEE-754 32-bit little-endian payloads.
- CRC fields are CRC-32, little-endian on the wire.
- CRC algorithm: reflected CRC-32 with polynomial 0xEDB88320, initial value 0xFFFFFFFF, final XOR 0xFFFFFFFF.

3.5 Configuration Mode Frames

3.5.1 1. Header Frame

The host sends one header frame first.

Header size: 18 bytes

Header Frame

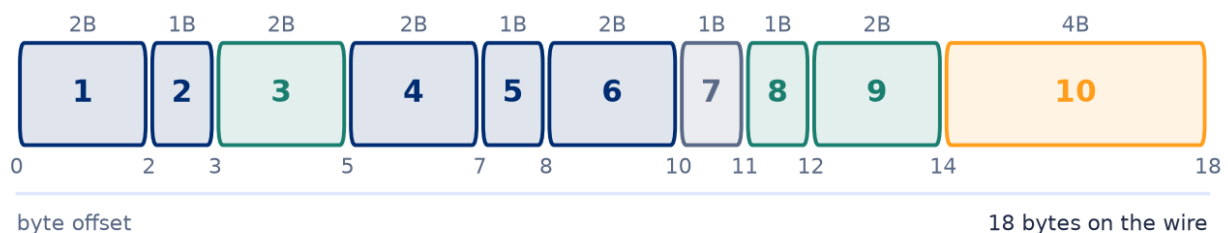


Figure 3.1: Header Frame, 18 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	2	sequence number
2	1	opcode = PP_CMD_WRITE (0x00)
3	2	body size in bytes
5	2	block ID
7	1	function ID
8	2	sub-function ID
10	1	number of parameters
11	1	number of body chunks

Offset	Size	Field
12	2	last chunk size
14	4	header CRC32

Header CRC coverage: bytes 0..13

3.5.2 2. Body Stream

The body is a logical byte stream that may be transmitted in one or more chunks.

Limits in the current implementation:

- body chunk size: 400 bytes (PP_CHUNK_MAX_SIZE)
- hard maximum body size: 65 535 bytes - the firmware buffer PP_CMD_MAX_SIZE is 65 536u (one byte of headroom)
- maximum parameter count: 20
- V2 batch upload buffer: PP_BATCH_BUFFER_SIZE = 65 536 bytes

The body stream layout is:

1. parameter descriptor for parameter 0
2. raw parameter bytes for parameter 0
3. parameter descriptor for parameter 1
4. raw parameter bytes for parameter 1
5. repeat for all parameters
6. body CRC32

Each parameter descriptor is a 16-bit little-endian word:

- bits 15:12: parameter type ID
- bits 11:0: parameter payload size in bytes

The host computes the descriptor as:

`descriptor = (type_id << 12) | payload_size`

The following parameter types are used by the current library:

- TYPE_String
- TYPE_RealArray
- TYPE_Uint32Array
- TYPE_Uint16Array
- TYPE_Uint8Array
- TYPE_Uint32
- TYPE_Real
- TYPE_Uint16
- TYPE_Uint8

Body CRC coverage: the entire descriptor-plus-payload stream, not including the 4 CRC bytes themselves.

Important detail: DataSize in the header is the body size including the body CRC. For a command with no parameters, DataSize is 4.

3.5.3 3. Response Poll Frame

After sending the header and all body chunks, the host does not immediately read a dedicated response frame. Instead it repeatedly transmits a response-request frame until a valid response appears.

Response-request size: 9 bytes

Response Poll Frame

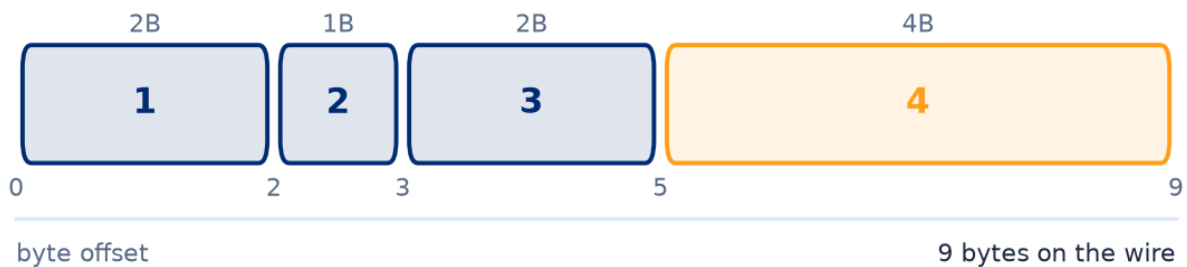


Figure 3.2: Response Poll Frame, 9 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	2	sequence number
2	1	opcode
3	2	fixed response ID 0xFFFF
5	4	CRC32

Normal configuration commands use:

- opcode PP_CMD_READ (0x01)

NewVectorProbe creation is a library special case and uses:

- opcode PP_CMD_READ_VECT_PROBE (0x02)

Response-request CRC coverage: bytes 0..4

3.5.4 4. Response Frame

When the board has a response ready, it returns a 7-byte response frame.

Response Frame



Figure 3.3: Response Frame, 7 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	2	sequence number
2	1	design status code
3	4	CRC32

Response CRC coverage: bytes 0..2

The host polls every 2 ms and times out after 1000 ms.

3.6 Function Classes

The header carries a top-level function class in FuncId.

- Createblock = 0x00
- Linkblock = 0x01
- Manageblock = 0x02

The actual operation is selected by SubFuncId.

These values are the block-creation contract. **They are tied to the block ABI version**, which a host reads from GetBoardInfo: a board of a different block ABI will build a different block from the same number. Read the ABI before sending any Create frame, and refuse to deploy on a mismatch rather than building the wrong diagram.

3.6.1 Create range, 0x0000 to 0x0FFF

One sub-function per block type. The complete set, 247 entries:

Name	Value
NewSys	0x0000 (0)

Name	Value
NewSysLightC	0x0001 (1)
NewSysLightD	0x0002 (2)
NewInput	0x0003 (3)
NewHwInput	0x0004 (4)
NewHwOutput	0x0005 (5)
NewNode	0x0006 (6)
NewFlexNode	0x0007 (7)
NewMacro	0x0008 (8)
NewProbe	0x0009 (9)
NewVectorProbe	0x000A (10)
NewSine	0x000B (11)
NewTri	0x000C (12)
NewSqr	0x000D (13)
NewSaw	0x000E (14)
NewNoise	0x000F (15)
NewSineAm	0x0010 (16)
NewButterLow	0x0011 (17)
NewButterHigh	0x0012 (18)
NewButterBand	0x0013 (19)
NewButterStop	0x0014 (20)
NewBiquadButterLow	0x0015 (21)
NewBiquadButterHigh	0x0016 (22)
NewBiquadButterBand	0x0017 (23)
NewBiquadButterStop	0x0018 (24)
NewChebyshev1Low	0x0019 (25)
NewChebyshev1High	0x001A (26)
NewChebyshev1Band	0x001B (27)
NewChebyshev1Stop	0x001C (28)
NewBiquadCheby1Low	0x001D (29)
NewBiquadCheby1High	0x001E (30)
NewBiquadCheby1Band	0x001F (31)
NewBiquadCheby1Stop	0x0020 (32)
NewChebyshev2Low	0x0021 (33)
NewChebyshev2High	0x0022 (34)
NewChebyshev2Band	0x0023 (35)

Name	Value
NewChebyshev2Stop	0x0024 (36)
NewBiquadCheby2Low	0x0025 (37)
NewBiquadCheby2High	0x0026 (38)
NewBiquadCheby2Band	0x0027 (39)
NewBiquadCheby2Stop	0x0028 (40)
NewEllipLow	0x0029 (41)
NewEllipHigh	0x002A (42)
NewEllipBand	0x002B (43)
NewEllipStop	0x002C (44)
NewBiquadEllipLow	0x002D (45)
NewBiquadEllipHigh	0x002E (46)
NewBiquadEllipBand	0x002F (47)
NewBiquadEllipStop	0x0030 (48)
NewNotch	0x0031 (49)
NewPeak	0x0032 (50)
NewAllPass	0x0033 (51)
NewFirLow	0x0034 (52)
NewFirHigh	0x0035 (53)
NewFirBand	0x0036 (54)
NewFirStop	0x0037 (55)
NewCustFir	0x0038 (56)
NewParamEQLow	0x0039 (57)
NewParamEQHigh	0x003A (58)
NewParamEQBand	0x003B (59)
NewParamEQPeak	0x003C (60)
NewCompressor	0x003D (61)
NewDecibelsConverter	0x003E (62)
NewCombFeedForward	0x003F (63)
NewCombFeedBack	0x0040 (64)
NewInterpolation	0x0041 (65)
NewMovingAverage	0x0042 (66)
NewDelayOp	0x0043 (67)
NewForwardFFT	0x0044 (68)
NewForwardFFT_HannWindow	0x0045 (69)
NewFftResolver	0x0046 (70)

Name	Value
NewPhaseResolver	0x0047 (71)
NewDWT	0x0048 (72)
NewIDWT	0x0049 (73)
NewAbsMax	0x004A (74)
NewAbsMin	0x004B (75)
NewEntropy	0x004C (76)
NewPower	0x004D (77)
NewMean	0x004E (78)
NewMaximum	0x004F (79)
NewMinimum	0x0050 (80)
NewRootMeanSquare	0x0051 (81)
NewStandardDeviation	0x0052 (82)
NewVariance	0x0053 (83)
NewPeriodMeter	0x0054 (84)
NewUpDwnPeriodMeter	0x0055 (85)
NewMag2	0x0056 (86)
NewMag3	0x0057 (87)
NewPid	0x0058 (88)
NewTunePID	0x0059 (89)
NewTunePI	0x005A (90)
NewTunePD	0x005B (91)
NewTune2PID	0x005C (92)
NewPidAw	0x00F2 (242)
NewTunePIDAw	0x00F3 (243)
NewTunePIAw	0x00F4 (244)
NewTune2PIDAw	0x00F5 (245)
NewPidFromModel	0x00F6 (246)
NewComp	0x005D (93)
NewSchmidt	0x005E (94)
NewSat	0x005F (95)
NewDeadZone	0x0060 (96)
NewBacklashDeadBand	0x0061 (97)
NewSign	0x0062 (98)
NewGain	0x0073 (115)
NewSqrMath	0x0074 (116)

Name	Value
NewSqrt	0x0075 (117)
NewSineFunc	0x0076 (118)
NewCosineFunc	0x0077 (119)
NewAbsVal	0x0078 (120)
NewAtan2	0x0079 (121)
NewLogFunc	0x007A (122)
NewExpFunc	0x007B (123)
NewNegate	0x007C (124)
NewInverse	0x007D (125)
NewSwitch	0x007E (126)
NewQuantizer	0x007F (127)
NewIntegrator	0x0080 (128)
NewDerivative	0x0081 (129)
NewRateLimiter	0x0082 (130)
NewSampleHold	0x0083 (131)
NewClarke	0x0084 (132)
NewInvClarke	0x0085 (133)
NewPark	0x0086 (134)
NewInvPark	0x0087 (135)
NewLeadLag	0x0088 (136)
NewWashout	0x0089 (137)
NewStateSpace	0x008A (138)
NewCompFilter	0x008B (139)
NewSMO	0x008C (140)
NewPMSM_EKF	0x008D (141)
NewAHRS_EKF	0x008E (142)
NewBatterySOC_EKF	0x008F (143)
NewTilt_EKF	0x0090 (144)
NewLoadTorque_EKF	0x0091 (145)
NewThermal_EKF	0x0092 (146)
NewBaroAlt_EKF	0x0093 (147)
NewRobotLoc_EKF	0x0094 (148)
NewSinTrack_EKF	0x0095 (149)
NewGridSync_EKF	0x0096 (150)
NewACIM_EKF	0x0097 (151)

Name	Value
NewBatterySOH_EKF	0x0098 (152)
NewWMR_EKF	0x0099 (153)
NewGPSINS_EKF	0x009A (154)
NewLMS	0x009B (155)
NewAdaptiveNotch	0x009C (156)
NewDOB	0x009D (157)
NewESC	0x009E (158)
NewADRC	0x009F (159)
NewMRAC	0x00A0 (160)
NewMRAC_Lyapunov	0x00A1 (161)
NewRepetitiveCtrl	0x00A2 (162)
NewSTR	0x00A3 (163)
NewILC	0x00A4 (164)
NewGainSchedulePID	0x00A5 (165)
NewGainScheduler	0x00A6 (166)
NewRelayAutoTune	0x00A7 (167)
NewStepAutoTune	0x00A8 (168)
NewSTR_PID	0x00A9 (169)
NewMRAC_PID	0x00AA (170)
NewESC_PID	0x00AB (171)
NewPerfMonitor_PID	0x00AC (172)
NewFuzzyPID	0x00AD (173)
NewMomentaryPower	0x00AE (174)
NewMaxObserver	0x00AF (175)
NewMinObserver	0x00B0 (176)
NewMux	0x00B1 (177)
NewDemux	0x00B2 (178)
NewLoudness	0x00B3 (179)
NewCpuLoad	0x00B4 (180)
NewExtSensor	0x00B5 (181)
NewLimiter	0x00B6 (182)
NewNoiseGate	0x00B7 (183)
NewEnvFollower	0x00B8 (184)
NewADSR	0x00B9 (185)
NewFracDelay	0x00BA (186)

Name	Value
NewChorus	0x00BB (187)
NewFlanger	0x00BC (188)
NewPhaser	0x00BD (189)
NewReverb	0x00BE (190)
NewStereoPan	0x00BF (191)
NewDistortion	0x00C0 (192)
NewBitcrusher	0x00C1 (193)
NewRingMod	0x00C2 (194)
NewPitchDetect	0x00C3 (195)
NewVUMeter	0x00C4 (196)
NewStereoMixer	0x00C5 (197)
NewDwtDenoise	0x00C6 (198)
NewDwtSubbandGain	0x00C7 (199)
NewDwtSubbandEnergy	0x00C8 (200)
NewSignalSource	0x00CA (202)
SetSignalSourceParams	0x00CB (203)
SignalSourceManualTrigger	0x00CC (204)
NewQuatMult	0x00CD (205)
NewWindowFramer	0x00CE (206)
NewStftFramer	0x00CF (207)
NewImuBiasEst	0x00D4 (212)
NewFeatureStack	0x00D5 (213)
NewZScoreNorm	0x00D6 (214)
NewInnovationChi2	0x00D7 (215)
NewMadgwickAhrs	0x00D8 (216)
NewMahonyAhrs	0x00D9 (217)
NewComplementaryAhrs	0x00DA (218)
NewMixerMultirotor	0x00DB (219)
NewLqrServo	0x00DC (220)
NewTrajScurve	0x00DD (221)
NewTrajTrapezoid	0x00DE (222)
NewFrictionStribeck	0x00DF (223)
NewInputShaper	0x00E0 (224)
NewHjorth	0x00E1 (225)
NewEmgEnvelope	0x00E2 (226)

Name	Value
NewEmgHudgins	0x00E3 (227)
NewEcgBlRemove	0x00E4 (228)
NewHrvTime	0x00E5 (229)
NewEegBandpower	0x00E6 (230)
NewSkewKurt	0x00E7 (231)
NewPercentile	0x00E8 (232)
NewPeakToPeak	0x00E9 (233)
NewCrestFactor	0x00EA (234)
NewZcr	0x00EB (235)
NewSpectralFeatures	0x00EC (236)
NewCepstrum	0x00ED (237)
NewAutocorrLags	0x00EE (238)
NewPcaProject	0x00EF (239)
NewDecisionTree	0x00F0 (240)
NewQuatFromEuler	0x00D0 (208)
NewQuatToEuler	0x00D1 (209)
NewQuatToRotmat	0x00D2 (210)
NewQuatNormConj	0x00D3 (211)
NewAdaptiveKF	0x0063 (99)
NewIdealKF	0x0064 (100)
NewRLS	0x0065 (101)
NewRRLS	0x0066 (102)
NewRLSMacro	0x00F1 (241)
NewMerger	0x0067 (103)
NewExpander	0x0068 (104)
NewSingleAnIn	0x0069 (105)
NewListOfAnIn	0x006A (106)
NewSinglePwmIn	0x006B (107)
NewListOfPwmIn	0x006C (108)
NewSinglePwmOut	0x006D (109)
NewListOfPwmOut	0x006E (110)
NewAccelerometer	0x006F (111)
NewGyroscope	0x0070 (112)
NewMagnetometer	0x0071 (113)
NewMotor	0x0072 (114)

Name	Value
NewEncoder	0x00C9 (201)

3.6.2 Link range, 0x1000 to 0x1FFF

The link sub-function is chosen by the KINDS of the two endpoints, not by their names. The complete set, 17 entries:

Name	Value
LinkSys2Sys	0x1000 (4096)
LinkNode2Node	0x1001 (4097)
LinkSys2Node	0x1002 (4098)
LinkIn2Node	0x1003 (4099)
LinkNode2Probe	0x1004 (4100)
LinkSys2Probe	0x1005 (4101)
LinkIn2Sys	0x1006 (4102)
LinkNode2Sys	0x1007 (4103)
LinkMacro2Macro	0x1008 (4104)
LinkMacro2Sys	0x1009 (4105)
LinkSys2Macro	0x100A (4106)
LinkMacro2Node	0x100B (4107)
LinkNode2Macro	0x100C (4108)
LinkIn2Macro	0x100D (4109)
LinkMacro2Probe	0x100E (4110)
LinkSys2VectorProbe	0x100F (4111)
LinkNode2VectorProbe	0x1010 (4112)

3.6.3 Manage Range

Manage sub-functions start at 0x2000.

The current library uses:

- DesignInit = 0x2000
- SetInput = 0x2001
- StopDesign = 0x2002
- StartMacro = 0x2003
- StopMacro = 0x2004
- StartSampling = 0x2005
- StopSampling = 0x2006
- EnableRemoteInput = 0x2007

- SetMonInterface = 0x2008
- GetDesignInfo = 0x2009 - query monitoring metadata (returns text payload)

3.6.3.1 EEPROM Application Storage

- EnableStorage = 0x200E - begin recording commands
- CommitToEeprom = 0x200F - finalize and write to EEPROM
- LoadApp = 0x2010 - load and replay stored app
- DumpApp = 0x2011 - download stored app data (returns data payload)
- UploadApp = 0x2012 - upload app data
- ListApps = 0x2013 - list stored apps (returns data payload)
- DeleteApp = 0x2014 - delete stored app
- SyncEeprom = 0x2015 - flush pending EEPROM writes

3.6.3.2 System Reset

- SoftReset = 0x2017 - soft reset: stop sampling, free blocks, stay connected

3.6.3.3 Offline Monitoring

- SetMonitorMode = 0x2018 - set monitoring mode. Params: uint8 PP_MonitorMode
 - 0 = Live (decimated scalar batch stream over USB)
 - 1 = Offline (full-rate capture to SDRAM, download after stop)
 - 2 = Dual (both live stream and offline capture). **Deprecated in Studio:** the Dual radio was retired from the Studio UI; the wire enum value still exists for binary compatibility, but the Studio Monitor view does not select it. Third-party hosts may still send 2 directly.
- SetTraceSize = 0x2019 - configure offline trace depth. Params: uint32 totalSamples
- SetLiveRate = 0x201A - set decimated live update rate. Params: uint16 scalarHz
- GetCaptureStatus = 0x201B - returns {captured:u32, total:u32, complete:u8}
- DownloadTrace = 0x201C - host pulls SDRAM trace. Params: uint32 offset, uint32 length

3.6.3.4 Boot-time application binding

- SetBootTestApp = 0x201D - params (uint16 AppId, uint8 AutoLoad); persist a boot-time application id
- GetBootTestApp = 0x201E - returns {AppId:u16, AutoLoad:u8}
- EraseAll = 0x201F - erase the entire EEPROM application directory (no params)

3.6.3.5 Board identity / authentication / bootloader entry

- GetBoardInfo = 0x2020 - returns {PartNumber:u16, UID:12 bytes} (14-byte payload)
- DescribeVectProbes = 0x2021 - returns the active design's vector-probe descriptor (header + per-probe records)
- Authenticate = 0x2022 - params Uint8Array nonce[32]; returns Uint8Array hmac[32] (HMAC-SHA256 over nonce || UID)

3.6.3.6 Storage / debug control

- `AbortStorage = 0x2024` - abort an in-progress `EnableStorage` recording without committing
- `GetDebugLog = 0x2025` - returns the firmware debug-log text payload
- `ClearDebugLog = 0x2026` - clear the firmware debug-log buffer
- `MeasureCycleTime = 0x2027` - returns 17-byte payload ($4 \times u32 + u8$) measuring the current design's per-cycle CPU profile

3.6.3.7 DC-motor capture

- `StartDcMotorCapture = 0x2028` - begin high-rate motor-ID capture; legacy 11-param and extended 15-param variants
- `GetDcMotorCaptureStatus = 0x2029` - returns 24-byte status {`captured:u32`, `target:u32`, `complete:u8`, `active_motors:u8`, `bytes_per_sample:u16`, `actual_fs_hz:f32`, `flags:u32`, `reserved:u32`}
- `DownloadDcMotorCapture = 0x202A` - host pulls captured samples; params (`u32` offset, `u32` length)
- `StopDcMotorCapture = 0x202B` - abort capture
- `GetDcMotorCaptureBoardState = 0x202C` - returns 4-byte {`busy:u8`, `reason:u8`, `reserved:u16`}

3.6.3.8 Session-1 FreqIn controls

- `SetFreqInCh2Source = 0x2030` - params (`u8`, `u8`); reroute FreqIn channel 2 source
- `SetFreqInSwapPins = 0x2031` - params (`u8`); swap FreqIn input pins

3.6.3.9 Deprecated Legacy Commands

- `ReadProbes = 0x200A` - use sampling-mode opcode `PP_CMD_READ` instead
- `ReadVectProbes = 0x200B` - use sampling-mode opcode `PP_CMD_READ_VECT_PROBE` instead
- `WriteInputs = 0x200C` - use sampling-mode opcode `PP_CMD_WRITE` instead
- `SwitchToCfg = 0x200D` - use sampling-mode opcode `PP_CMD_SWITCH_BACK_TO_CFG` instead

3.7 Sampling Mode Commands

After a successful `StartSampling`, the host switches to a simpler data-exchange protocol.

3.7.1 Sampling Mode Opcodes

Opcode	Name	Direction
0x00	PP_CMD_WRITE	host → board (remote inputs)
0x01	PP_CMD_READ	board → host (scalar probes)
0x02	PP_CMD_READ_VECT_PROBE	board → host (vector probes)

Opcode	Name	Direction
0x03	PP_CMD_SWITCH_BACK_TO_CFG	host → board
0x04	PP_CMD_FLOW_CREDIT	host → board (V2 flow control)

3.7.2 Write Remote Inputs

1. host sends one byte: PP_CMD_WRITE (0x00)
2. host sends a second SPI frame containing raw input values

Payload format:

- contiguous float32 little-endian values
- payload size = remote_input_count * 4

There is no sequence number and no CRC in this sampling-mode transfer.

3.7.3 Read Scalar Probes

1. host sends one byte: PP_CMD_READ (0x01)
2. host starts a second SPI frame and clocks out probe bytes by sending dummy 0x00

Payload format returned by the board:

- contiguous float32 little-endian values
- payload size = probe_count * 4

3.7.4 Read Vector Probes

1. host sends one byte: PP_CMD_READ_VECT_PROBE (0x02)
2. host waits 2 ms
3. host starts a second SPI frame and clocks out vector-probe bytes

Returned payload size:

- VectorProbeBytes
- in practice, the sum of all linked vector-probe sample bytes for the active board context

3.7.5 Switch Back To Configuration Mode

To leave sampling mode, the host sends one byte:

- PP_CMD_SWITCH_BACK_TO_CFG (0x03)

The current stopSampling() flow sends this byte first, then sends the normal managed StopSampling command in configuration mode.

3.8 Expected Board State Machine

The host implementation implies the following PAPAYA-board slave state machine.

3.8.1 Reset and Selection

For direct SPI targets:

1. host drives the board-select lines for the target board
2. host asserts reset
3. host initializes SPI
4. host releases reset
5. board enters configuration idle state

For USB-hosted targets:

1. host opens the USB bridge connection
2. bridge firmware owns local SPI and reset sequencing if required
3. the PAPAYA board enters configuration idle state through the bridge path

3.8.2 Configuration State Machine

CFG_IDLE

- wait for a valid header frame

CFG_WAIT_BODY

- receive NbrOfChunks body frames
- reassemble the body stream
- validate body CRC

CFG_EXECUTE

- decode FuncId
- decode SubFuncId
- parse parameter descriptors and payloads
- execute the requested create, link, or manage action
- prepare a response status for the current sequence number

CFG_RESPONSE_READY

- wait for a response-request frame with matching sequence number
- return the 7-byte response frame
- transition back to CFG_IDLE

3.8.3 Sampling State Machine

STREAM_IDLE

- entered after successful StartSampling

While in STREAM_IDLE:

- on PP_CMD_WRITE: transition to STREAM_WAIT_INPUT_PAYLOAD
- on PP_CMD_READ: transition to STREAM_TX_PROBES
- on PP_CMD_READ_VECT_PROBE: transition to STREAM_TX_VECTOR_PROBES
- on PP_CMD_SWITCH_BACK_TO_CFG: transition to CFG_IDLE

STREAM_WAIT_INPUT_PAYLOAD

- receive the input byte stream
- update remote inputs
- return to STREAM_IDLE

STREAM_TX_PROBES

- shift out the scalar probe byte stream
- return to STREAM_IDLE

STREAM_TX_VECTOR_PROBES

- prepare vector-probe bytes
- shift them out
- return to STREAM_IDLE

3.9 Host-Library Behavior

The current host implementation adds a few important behavioral rules.

3.9.1 Sequence Numbers

- sequence numbers are 16-bit
- they increment only after a successful configuration command
- sampling-mode transfers do not use sequence numbers

3.9.2 Error Handling

- configuration commands run only while `currentComStatus == ComOk` and `currentDesignStatus == DesignOk`
- on a timeout while polling for a response, the host sets `UnknownComError`
- the response parser validates sequence number and CRC before accepting the frame

Important implementation detail: there are TWO status enums and they are reported independently. `WrongSeqNumber`, `WrongCrc` and `UnknownComError` live in `PP_ComStatus`. `PP_DesignStatus`, whose value is the byte at offset 2 of every response payload, does not contain those names:

its members are `DesignOk`, `BadParam`, `BadConnection`, `BadRequest`, the per-peripheral failure codes (`GPIO_Failure` through `USB_Failure`) and `UndefinedFailure`. The host's `GetResponse()` path does not assign `WrongSeqNumber` / `WrongCrc` explicitly; a bad response simply keeps polling until timeout, then reports `UnknownComError`.

3.9.3 Monitoring Interface

The host can explicitly select:

- `MonitoringOverUsb`
- `MonitoringOverSpi`

SPI probe exchange is only meaningful when the board has been configured for SPI monitoring.

3.9.4 Multi-Board Sessions

The host keeps one runtime context per board ID 0..3.

That isolates:

- block-ID allocators
- sequence numbers
- design and communication status
- remote input queues
- scalar probe queues
- vector-probe buffers

On direct SPI targets, board selection is done in hardware through the two select lines and shared CS. On USB targets, the constructor still receives a board ID for API uniformity, but the active USB endpoint or bridge connection selects the target and the board ID has no hardware meaning.

3.10 Practical Transaction Summary

3.10.1 Configure Design

For direct SPI targets:

1. select board
2. send header
3. send body chunk 0
4. send remaining chunks if any
5. poll for response
6. if `StartSampling` succeeds, enter sampling mode

For USB targets:

1. select the target USB bridge or endpoint in the host application
2. send header
3. send body chunk 0

4. send remaining chunks if any
5. poll for response
6. if StartSampling succeeds, enter sampling mode

3.10.2 Stream Data

1. write inputs with PP_CMD_WRITE
2. read scalar probes with PP_CMD_READ
3. read vector probes with PP_CMD_READ_VECT_PROBE
4. send PP_CMD_SWITCH_BACK_TO_CFG before configuration-mode commands

3.11 V2 Protocol Extensions

V2 extends the base protocol with batch upload, auto-push framing, and flow control. V2 is negotiated per-session; unrecognized boards fall back to V1.

3.11.1 V2 Handshake

The host sends an 8-byte handshake packet in configuration mode:

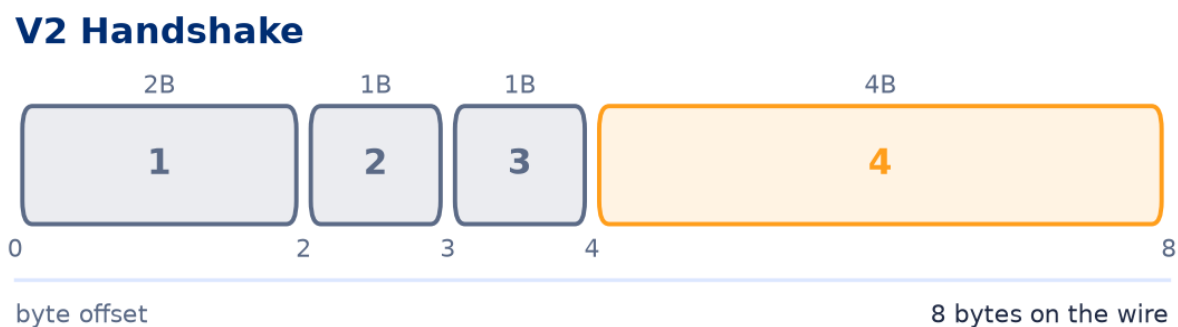


Figure 3.4: V2 Handshake, 8 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	2	magic 0x5041 ("PA")
2	1	version request (0x02)
3	1	flags (reserved, send 0x00)
4	4	CRC32 of bytes 0..3 (little-endian)

The firmware validates the CRC32; a mismatch causes the firmware to silently drop the request, so a host that pads bytes 3..7 with zeros will sit in handshake limbo. The board replies with the same magic plus its supported version and a 1-byte capabilities field, terminated by a CRC32 over the first four bytes.

3.11.2 V2 Batch Upload

Sends the entire design as a single transfer instead of one frame at a time.

Batch header (14 bytes):

V2 Batch Upload

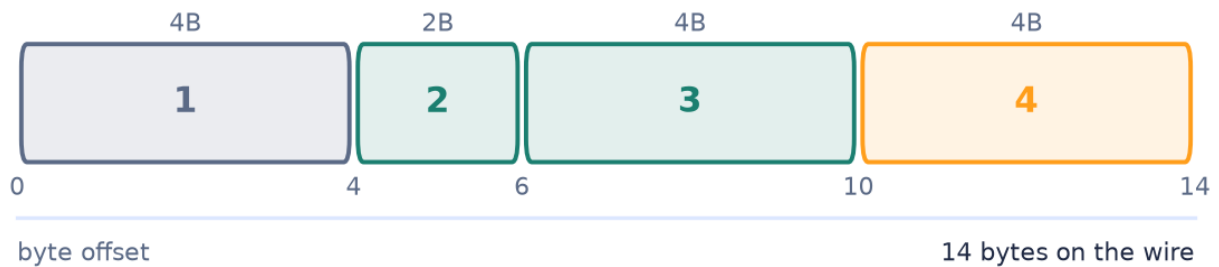


Figure 3.5: V2 Batch Upload, 14 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	4	magic 0x44455347 ("DESG")
4	2	frame count
6	4	payload byte count
10	4	payload CRC32

Payload: concatenated V1 frames (header + body for each command).

Batch response (13 bytes):

V2 Batch Upload

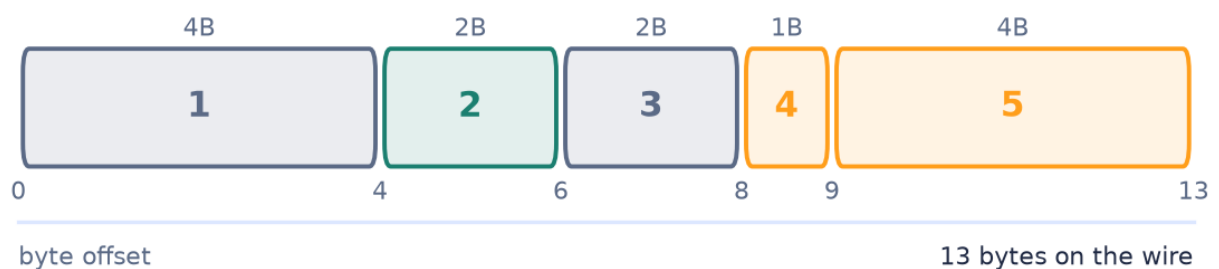


Figure 3.6: V2 Batch Upload, 13 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	4	magic 0x44455347 ("DESG")
4	2	frame count actually processed
6	2	last successfully-processed frame index
8	1	status: PP_DesignStatus of the failing frame, or 0 if all OK. The firmware also writes synthetic codes for batch-level errors: 0xFD = parse error, 0xFE = truncated frame, 0xFF = CRC mismatch - those values do not appear in the PP_DesignStatus enum.
9	4	CRC32 over bytes 0..8

3.11.3 V2 Auto-Push Frame

In V2, the board autonomously pushes probe data with sequence and CRC:

V2 Auto-Push Frame

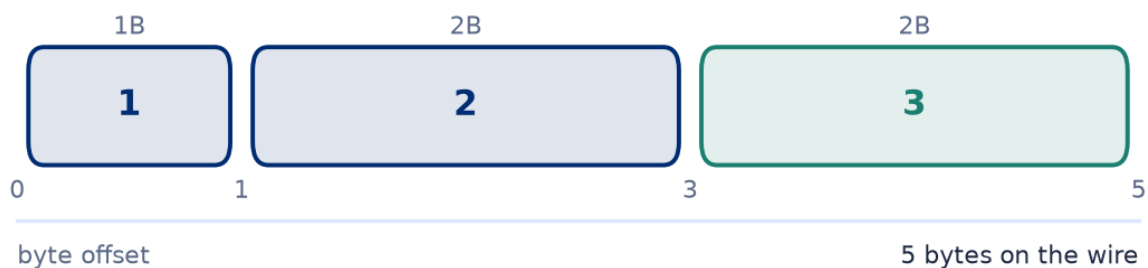


Figure 3.7: V2 Auto-Push Frame, 5 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	1	opcode (0x01 scalar, 0x02 vector)
1	2	sequence number
3	2	payload length
5	N	payload
5+N	4	CRC32 (covers bytes 0..4+N)

3.11.4 V2 Flow Control

The host grants TX credits to the board using opcode 0x04:

V2 Flow Control

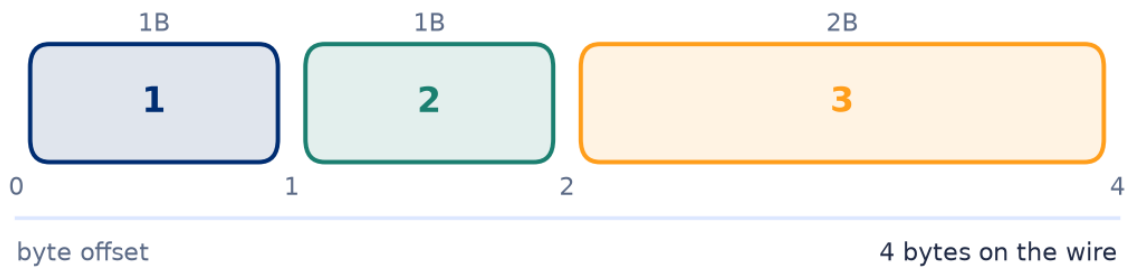


Figure 3.8: V2 Flow Control, 4 bytes on the wire. The numbered boxes are drawn to scale by byte count and key to the rows of the table below.

Offset	Size	Field
0	1	opcode 0x04
1	1	credit count (1-255)
2	2	CRC16

The board gates auto-push frames on `PP_TxCredits > 0`. Each transmitted frame decrements the counter. The host grants credits to control the data rate and prevent USB buffer overflow.

3.12 Protocol Limits

Parameter	Value
Max scalar probes	16 (see the note below)
Max inputs	16
Max vector probes	9
Max bytes per vector probe	8 192
Command header size	18 bytes
Response size	7 bytes
Max chunk size	400 bytes
Max body size	65 535 bytes (firmware buffer = 65 536)
V2 batch upload buffer	65 536 bytes
Max parameters per command	20
EEPROM max apps	16

Parameter	Value
EEPROM app label max size	128 bytes

The scalar-probe maximum above is a fixed array length in the firmware, and it is the same number at every sampling frequency. **There is no probe budget keyed on the sampling rate.** A design carrying 16 probes is accepted at 100 Hz and at 50 kHz alike.

What the sampling rate does decide is how much of that data the link can carry away. The board does not pre-refuse a design on that basis; it runs it and reports the CPU cost per sampling period, and a design that outruns the link loses samples rather than being rejected. A host that builds a design itself gets exactly the same treatment.

See *PAPAYA Studio - User Manual* for the refusal message the 17th probe produces and what to do about it.

3.12.1 Manage range, 0x2000 and above

The complete set, 66 entries. A gap in the numbering is deliberate and permanent: a removed opcode's number is never reused, because boards already in the field still speak it.

Name	Value
DesignInit	0x2000 (8192)
SetInput	0x2001 (8193)
StopDesign	0x2002 (8194)
StartMacro	0x2003 (8195)
StopMacro	0x2004 (8196)
StartSampling	0x2005 (8197)
StopSampling	0x2006 (8198)
EnableRemoteInput	0x2007 (8199)
SetMonInterface	0x2008 (8200)
GetDesignInfo	0x2009 (8201)
SwitchToCfg	0x200D (8205)
EnableStorage	0x200E (8206)
CommitToEeprom	0x200F (8207)
AbortStorage	0x2024 (8228)
GetDebugLog	0x2025 (8229)
ClearDebugLog	0x2026 (8230)
MeasureCycleTime	0x2027 (8231)
LoadApp	0x2010 (8208)
DumpApp	0x2011 (8209)

Name	Value
UploadApp	0x2012 (8210)
ListApps	0x2013 (8211)
DeleteApp	0x2014 (8212)
SyncEeprom	0x2015 (8213)
McuReset	0x2016 (8214)
SoftReset	0x2017 (8215)
SetMonitorMode	0x2018 (8216)
SetTraceSize	0x2019 (8217)
SetLiveRate	0x201A (8218)
GetCaptureStatus	0x201B (8219)
DownloadTrace	0x201C (8220)
SetBootApplication	0x201D (8221)
GetBootCfg	0x201E (8222)
EraseAll	0x201F (8223)
GetBoardInfo	0x2020 (8224)
DescribeVectProbes	0x2021 (8225)
Authenticate	0x2022 (8226)
EnterBootloader	0x2023 (8227)
StartDcMotorCapture	0x2028 (8232)
GetDcMotorCaptureStatus	0x2029 (8233)
DownloadDcMotorCapture	0x202A (8234)
StopDcMotorCapture	0x202B (8235)
GetDcMotorCaptureBoardState	0x202C (8236)
GetLiveDiagCounters	0x202D (8237)
SetFreqInCh2Source	0x2030 (8240)
SetFreqInSwapPins	0x2031 (8241)
GetExtSamplingDeadlineInfo	0x2032 (8242)
GetDesignInstrumentation	0x2033 (8243)
ListAudioSlots	0x2034 (8244)
GetAudioSlotInfo	0x2035 (8245)
EraseAudioSlot	0x2036 (8246)
EraseAllAudioSlots	0x2037 (8247)
DownloadAudioSlot	0x2038 (8248)
UploadAudioSlotStart	0x2039 (8249)
UploadAudioSlotChunk	0x203A (8250)

Name	Value
UploadAudioSlotCommit	0x203B (8251)
SetDwtSubbandGains	0x203C (8252)
SetDwtDenoiseScale	0x203D (8253)
ListDatasets	0x2040 (8256)
GetDatasetInfo	0x2041 (8257)
EraseDataset	0x2042 (8258)
UploadDatasetStart	0x2043 (8259)
UploadDatasetChunk	0x2044 (8260)
UploadDatasetCommit	0x2045 (8261)
DownloadDataset	0x2046 (8262)
SetMonBatchDepth	0x2060 (8288)
GetOfflineTraceGeometry	0x2070 (8304)

3.13 Parameter type identifiers

The descriptor is (type_id << 12) | payload_size, and these are the type identifiers it takes.

Name	Value
TYPE_String	0x0000 (0)
TYPE_RealArray	0x0001 (1)
TYPE_Uint32Array	0x0002 (2)
TYPE_Uint16Array	0x0003 (3)
TYPE_Uint8Array	0x0004 (4)
TYPE_Uint32	0x0005 (5)
TYPE_Real	0x0006 (6)
TYPE_Uint16	0x0007 (7)
TYPE_Uint8	0x0008 (8)
TYPE_NoType	0x00FF (255)

3.14 Design status values

The status byte of a response frame. DesignOk is the only success.

Name	Value
DesignOk	0x0000 (0)
MemError	0x0001 (1)
TfError	0x0002 (2)
UnknownC2dAlgo	0x0003 (3)
TextOk	0x0004 (4)
TxtError	0x0005 (5)
BadParam	0x0006 (6)
BadConnection	0x0007 (7)
BadRequest	0x0008 (8)
SamplingFreqOutOfLimit	0x0009 (9)
GPIO_Failure	0x000A (10)
TIM_Failure	0x000B (11)
ADC_Failure	0x000C (12)
DAC_Failure	0x000D (13)
COMP_Failure	0x000E (14)
DMA_Failure	0x000F (15)
OPAMP_Failure	0x0010 (16)
SPI_Failure	0x0011 (17)
UART_Failure	0x0012 (18)
I2C_Failure	0x0013 (19)
USB_Failure	0x0014 (20)
UndefinedFailure	0x0015 (21)
NoDesignStatus	0x00FF (255)

3.15 The 3-second configuration-idle watchdog

While in configuration mode, three seconds of silence ends the session. The board drops the configuration context and zeroes its command counter. The next command the host sends is then interpreted against an empty context, and the symptom is a design that reports **status 8** at StartSampling rather than any error at the point the timeout occurred.

Consequences for an implementation:

- Do not pause between Create commands to do slow work host-side. Build the whole command sequence first, then send it.

- A debugger breakpoint between two configuration commands will silently end the session. This is the single most common cause of a “the board stopped responding” report during bring-up.
- If you must pause, restart the sequence from DesignInit. There is no resume.

3.16 Command ordering

- DesignInit must precede every Create. It sets the sampling frequency and the discretisation algorithm, and **continuous blocks are discretised at the moment they are created**, using whatever those globals hold then.
- A Link may only name blocks that a previous Create has instantiated.
- A block joins the execution array when its **last output** is connected – not when it is created. This is why a hardware output actuates one sampling period behind the block that produced its value.

3.17 Mode transitions

- Configuration commands sent while in sampling mode are not understood, and the board will not report a protocol error for them.
- Return to configuration mode with the explicit switch-back command before attempting any further structured command.

3.18 What the protocol does not protect you from

The board validates what it can, but a structurally valid design can still be wrong. A link to the wrong port index, a block created with parameters outside its useful range, or a sampling frequency the design cannot meet will all be accepted. **The protocol reports transport and structural failures, not modelling mistakes.**

Chapter 4

Implementation checklist

Working through these in order is the shortest path to a first successful session.

1. **Transport.** Byte in, byte out, at the documented clock and mode. Prove it with a command whose response you can predict before attempting a design.
2. **CRC.** Reflected CRC-32, polynomial 0xEDB88320, initial 0xFFFFFFFF, final XOR 0xFFFFFFFF, little-endian on the wire. Verify against a known vector before trusting any frame.
3. **Endianness.** Every integer field and every float is little-endian. A byte-swapped float is the classic first bug: the board accepts it, and the design merely behaves strangely.
4. **One command, one response.** Poll for the response before sending the next command. Do not pipeline configuration commands.
5. **Respect the 3-second window.** Assemble the whole sequence before you start sending it.
6. **Then, and only then, a design.** DesignInit, two blocks, one link, StartSampling, read the probe.

Chapter 5

Glossary

Term	Meaning
Block	One element of the diagram – a filter, a gain, an input, a probe
Link	A connection from one block's output to another's input
Probe	A block that exposes a value for the host to read
Input	A block holding a constant the host may write at run time
Configuration mode	Structured, acknowledged commands; the design is being built
Sampling mode	The design is executing; short streaming commands only
Design	The complete set of blocks and links currently on the board